AI Engineering and Claude Architect · Part I, Foundations

AI and ML Fundamentals, Lesson 3

Neural Networks and How They Train

A linear model looks at "new device, or new country, but not both" and gives up, answering 50% for everything. Add one small hidden layer, and it gets all four cases right. This lesson builds that network by hand and shows why.

Module	AI01 of 13
Lesson	3 of 4
Prerequisites	AI01 L1–L2. Basic calculus helps, and the slope idea is explained
Exam relevance	Out of scope. Needed to understand AI02 and advanced interviews
Reading time	~45 minutes

Contents

1. Neural Networks and How They Train

Lesson 3 of 4 · AI and ML Fundamentals

Neural Networks and How They Train

Key idea

A neural network is logistic regression stacked in layers, and training is one repeated move: measure the error, trace it back through every weight, nudge each weight downhill. Non-linear activation functions let layers build complex patterns from simple ones. Backpropagation makes it affordable to compute every weight's share of the error. Almost everything else, from Adam to dropout to residual connections, **exists to keep this process stable and to stop it from memorizing.**

> *Before: logistic regression, overfitting (AI01 L2). Teaches: neurons, activations, loss functions, backpropagation, optimizers, learning rates, vanishing gradients, regularization, CNNs. Used by: AI02 L2–L4 (transformers and how LLMs train), AI11 L1 (fine-tuning).*

The hook: the pattern a straight line can't learn

Your risk team notices something. A customer with a **new device** alone is suspicious. So is a customer in a **new country**. But both together usually means someone traveling with a new phone, which is normal.

New device	New country	Risky?
No	No	No
No	Yes	Yes
Yes	No	Yes
Yes	Yes	No

This pattern is called **XOR** (exclusive or). You train logistic regression for 20,000 rounds. Real output, from `code/ai01-ml/TinyNeuralNet.java`:

```
linear model predictions: 0.50 0.50 0.50 0.50
```

Key idea

It doesn't improve with more training or data, because the shape of the model is wrong. Logistic regression draws a single straight boundary. No straight line can put the two "yes" cases on one side and the two "no" cases on the other. The best it can do here is answer 50% for everything.

Interactions like this are everywhere in real data, and in language. The word "not" flips the meaning of the next word, which is exactly this kind of interaction.

Intuition: a committee of specialists

Imagine a chain of analysts. The first group each looks for one simple thing: "is it a new device?", "is it a new country?". The next group reads their reports and spots combinations: "exactly one of those fired". A final analyst combines those findings into a decision.

Nobody told them what to look for. They started out making random judgments, and each time the final decision was wrong, the blame was passed backward, and each analyst adjusted in proportion to their share of the blame.

That's a neural network, and that blame-passing is backpropagation.

The building blocks

A neuron

One neuron does three things: computes a weighted sum, adds a **bias** (a constant offset), and applies an **activation function**.

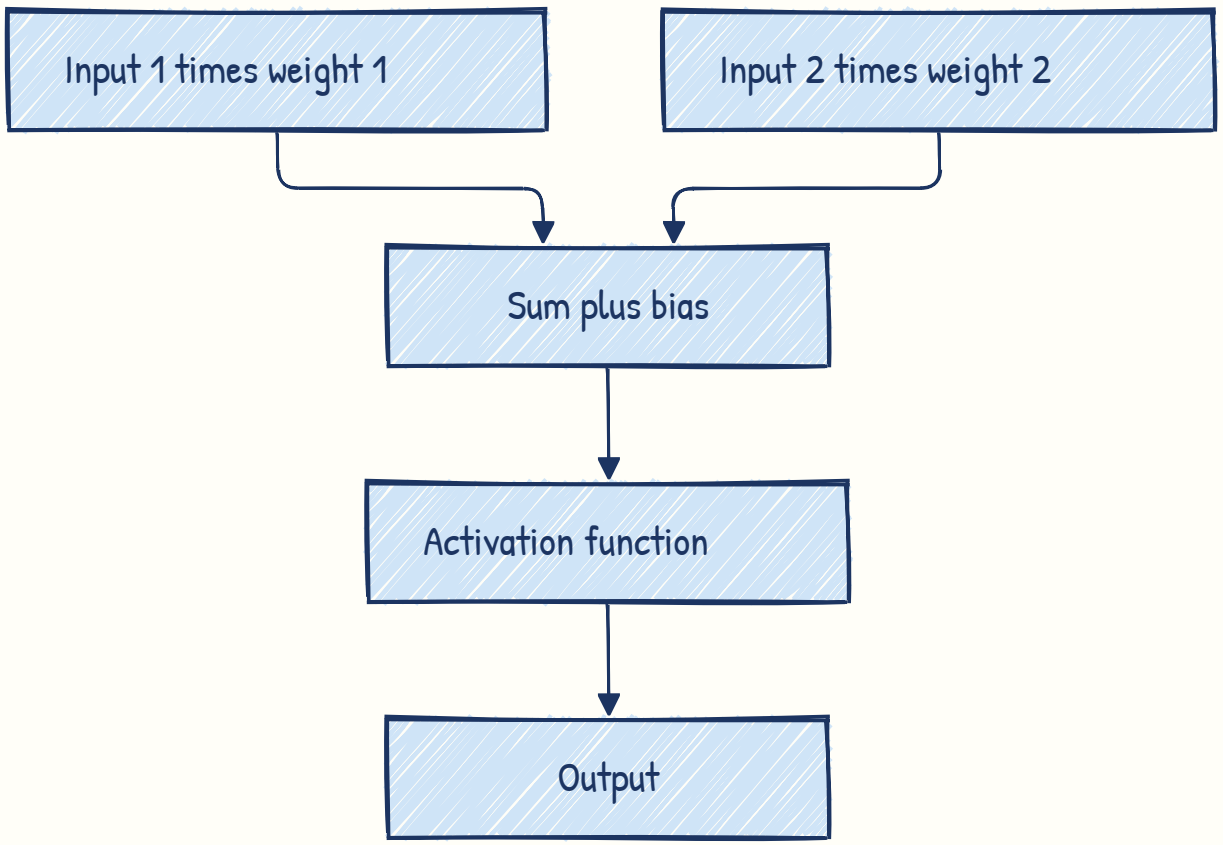


Figure 1. A single neuron. It's logistic regression when the activation is a sigmoid.

Why activations must be non-linear

Stack linear functions, and you get one linear function. `2(3x)` is simply `6x`. A hundred such layers would still draw a single straight line. The non-linear activation is what makes depth useful.

Activation	Output range	Where it's used	Weakness
Sigmoid	0 to 1	Binary output layer	Flat at the extremes, so gradients vanish
Tanh	−1 to 1	Hidden layers in older and small networks	Same flatness
ReLU	0 and up	Common default in hidden layers	A neuron can get stuck at zero
Leaky ReLU	Any	Reduces that stuck-neuron problem	One more choice to make
GELU	Mostly 0 and up	A smooth ReLU, used in many transformers	Slightly more compute
Softmax	Probabilities summing to 1	Output choosing among many options	Only for output layers

Softmax is the one to remember. An LLM's last step is a softmax over its whole vocabulary, which is what produces next-token probabilities (AI02 L5).

Layers

With a **hidden layer** between input and output, each hidden neuron learns its own boundary, and the output combines them. Our network has 2 inputs, 4 hidden neurons and 1 output:

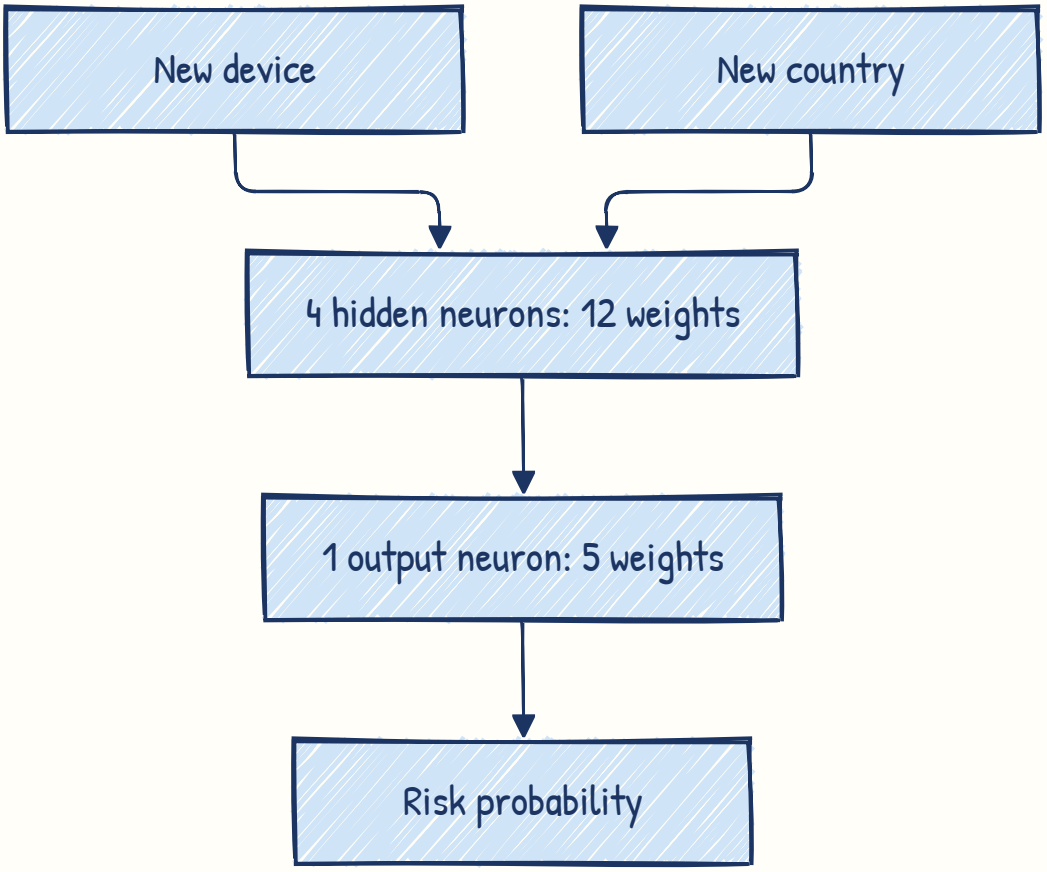


Figure 2. The small network in this lesson. Four hidden neurons with three weights each, plus five output weights, make 17 parameters.

That's 17 parameters. Real output after training:

```
network predictions: 0.00 1.00 1.00 0.00
```

All four cases are now correct. The model that gave up at 50% differs only in that it has a hidden layer.

Training: four repeated steps

Step 1: Forward pass

Run the input through the layers to get a prediction.

```
double forward(double[] x) {
    double[] h = hidden(x);
    double z = outputWeights[HIDDEN];
    for (int i = 0; i < HIDDEN; i++) {
        z += outputWeights[i] * h[i];
    }
    return sigmoid(z);
}
```

Step 2: Measure the loss

A **loss function** turns "how wrong" into one number.

Task	Common loss	Behavior
Predict a number	Mean squared error	Large errors are penalized heavily
Predict a category	**Cross-entropy**	Being confidently wrong is penalized heavily

Cross-entropy is `-ln(p)`, where `p` is the probability given to the correct answer. Giving the right answer 0.9 costs 0.105. Giving it 0.01 costs 4.6. **LLMs train using this same loss**, applied to next-token predictions.

Step 3: Backpropagation

We need to know how much each weight contributed to the loss, that is, the slope of the loss with respect to that weight. This slope is called the **gradient**.

The obvious way is to nudge each weight, then rerun the network. That takes one full pass per weight, which is impossible with billions. Backpropagation, popularized in a 1986 Nature paper, gets every gradient in about one backward pass, by applying the chain rule from the output back through each layer. Results computed for later layers get reused for earlier ones.

> *Key idea*
>
> **Backpropagation is why large networks are trainable at all.** It turns the cost of finding all gradients from "one pass per weight" into roughly "one extra pass".

A worked example with one neuron. Take input `x = 2`, weight `w = 0.5`, no bias, target `y = 1`, and learning rate `0.1`.

Step	Calculation	Value
Weighted sum	`z = 0.5 × 2`	1.000
Prediction	`p = σ(1)`	0.7311
Loss	`-ln(0.7311)`	0.3133
Gradient	`(p - y) × x = (0.7311 - 1) × 2`	−0.5379
Update	`w = 0.5 - 0.1 × (-0.5379)`	0.5538
New prediction	`σ(0.5538 × 2)`	0.7517
New loss	`-ln(0.7517)`	0.2854

The loss fell from 0.3133 to 0.2854. That is one step, and training repeats it billions of times.

The same chain rule runs in our network:

```
double error = forward(x) - TARGETS[n];
for (int i = 0; i < HIDDEN; i++) {
    g.output[i] += error * h[i];
    double slope = 1 - h[i] * h[i];            // derivative of tanh
    double common = error * outputWeights[i] * slope;
    g.hidden[i][0] += common;
    g.hidden[i][1] += common * x[0];
    g.hidden[i][2] += common * x[1];
}
```

Line by line, the idea is that `common` carries the blame backward: output error, times how much this neuron mattered, times how sensitive this neuron was. You can check that it's right by comparing it with the slow nudge method:

```
gradient check: backprop=-0.02774739 numeric=-0.02774739
```

Frameworks like PyTorch automate this, a feature called **automatic differentiation**, so nobody writes this by hand in practice.

Step 4: Update using gradient descent

Move each weight a small step against its gradient:

```
weight = weight - learningRate × gradient
```

Picture standing on a foggy hillside. You can't see the valley, but you can feel which way is downhill under your feet. You take a step and feel again.

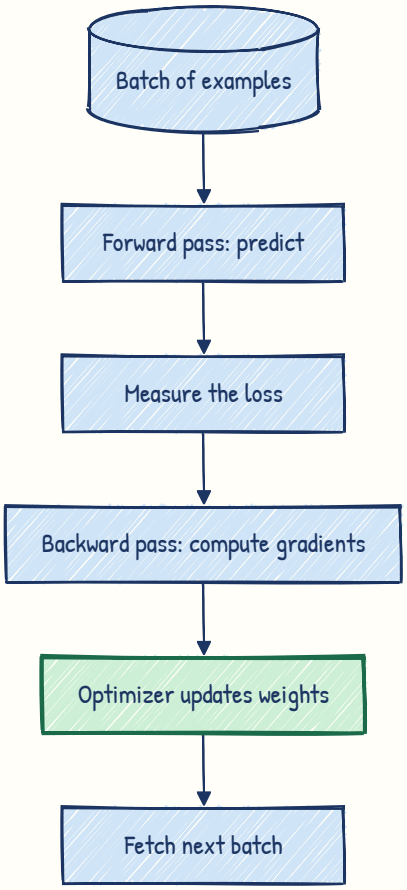


Figure 3. One training step. Each trip around this loop uses one batch of examples, and large models repeat it an enormous number of times.

The learning rate: the setting that matters most

Same network, same data, three learning rates. Real output, showing loss at every 500th round:

```
lr=0.50  loss: 0.6853 0.0116 0.0052 0.0034 0.0025
lr=0.01  loss: 0.6853 0.6066 0.5381 0.4762 0.4064
lr=20.00 loss: 0.6853 3.7650 NaN 5.5467 3.4050
```

Rate	What happened	In the hillside picture
0.50	Fast, stable progress	Sensible steps
0.01	Still far from done after 2,000 rounds	Tiny shuffling steps
20.0	Loss rose, and even reached `NaN`	Leaping over the valley onto the other slope

> *That `NaN` is a real production signal*
>
> When a training run's loss turns to `NaN` or suddenly jumps, the usual first suspect is the learning rate. In this demo, `NaN` came from a probability saturating to exactly 0 or 1 and then taking its log. Production training pipelines watch for this and stop early rather than waste days of GPU time.

Optimizers: smarter ways to step

Computing gradients on all the data before each step is too slow, so training uses samples.

Variant	Uses	Trade-off
Batch gradient descent	All data per step	Accurate, slow
Stochastic gradient descent (SGD)	One example per step	Fast, very noisy
Mini-batch	Typically tens to thousands of examples	The usual choice, and fits GPU hardware

An **optimizer** decides how to use the gradient.

Optimizer	Idea	Analogy
SGD	Step directly against the gradient	Walking downhill
Momentum	Keep some speed from earlier steps	A ball rolling, smoothing out jitter
RMSProp	Scale each weight's step by its recent gradient size	Shorter strides on steep ground
Adam	Combines momentum and per-weight scaling	A common default
AdamW	Adam with better-behaved weight decay	Common for transformers

Learning rate schedules usually raise the rate gently at first, called warm-up, and then lower it over time so the model can settle.

What goes wrong, and the fixes

Vanishing and exploding gradients

Backpropagation multiplies slopes layer by layer. A sigmoid's slope is never above 0.25, so in a deep sigmoid network, the blame reaching early layers shrinks fast. Computed best case:

```
 2 sigmoid layers: best-case gradient factor = 6.25e-02
 5 sigmoid layers: best-case gradient factor = 9.77e-04
10 sigmoid layers: best-case gradient factor = 9.54e-07
20 sigmoid layers: best-case gradient factor = 9.09e-13
```

Early layers effectively stop learning. That's **vanishing gradients**. The opposite, where products grow out of control, is **exploding gradients**.

Fix	How it helps
ReLU-style activations	Slope stays at 1 for active neurons, so the product doesn't shrink
Careful weight initialization	Keeps signal sizes stable across layers
Normalization layers	Keep each layer's values in a stable range. Transformers typically use layer normalization
Residual connections	Add a layer's input to its output, giving gradients a direct path. Transformers use them throughout
Gradient clipping	Caps gradient size to stop explosions

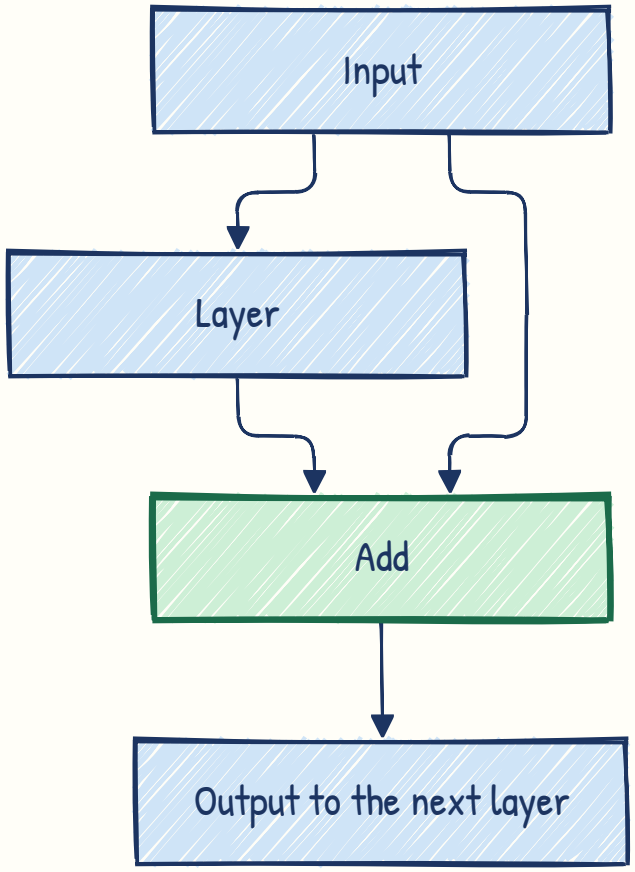


Figure 4. A residual connection. The input skips past the layer and is added back, so gradients have a direct route to earlier layers.

You'll see normalization and residual connections again when AI02 L3 assembles a transformer. They aren't decoration. Without them, networks that deep would be very hard to train.

Overfitting, and regularization

Bigger networks memorize more readily. **Regularization** means anything that discourages memorization.

Technique	How it works
L2 / weight decay	Penalizes large weights, favoring simpler solutions
L1	Pushes many weights to exactly zero
Dropout	Randomly turns off neurons during training, so none can be relied on alone
Early stopping	Stops when validation loss starts to rise
More data, or augmentation	Makes memorizing harder

Other architectures, briefly

Structure in the network should match structure in the data.

Architecture	Built for	Key idea
Fully connected	Generic features	Every neuron sees all of the previous layer
CNN (convolutional)	Images	A small filter slides across the image, reusing the same weights everywhere
RNN, LSTM, GRU	Sequences	Passes memory along step by step (AI01 L4)
Transformer	Sequences, especially language	Attention relates all positions directly (AI02)
Autoencoder	Compression	Learns to squeeze data and rebuild it (AI01 L4)

CNNs are worth one more sentence, because their insight recurs: a pattern such as an edge looks the same wherever it appears, so **sharing weights across positions** saves enormous numbers of parameters. Transformers share weights across positions too.

How the field solved it

Problem	How it was solved	Where it returns
Linear models can't capture interactions	Hidden layers with non-linear activations	Everywhere
Gradient computation was too expensive	Backpropagation	All LLM training
Deep networks wouldn't train	ReLU, normalization, residual connections	Transformers (AI02 L3)
Plain SGD is slow and unstable	Momentum, Adam	LLM training
Training took too long on CPUs	GPUs, as AlexNet showed in 2012	AI02 L4

This is shared field history, and it isn't specific to Anthropic. Where Anthropic comes in is at the top of this stack. AI02 L4 covers what it added on the alignment side, and AI02 L6 covers its work on looking inside trained networks.

The production architecture

This is a recommended reference design, not a specific company's system. You probably won't train large models. You may well serve a small one, or evaluate a vendor's. Training a large LLM is covered in AI02 L4.

Baseline: a single machine. A data scientist trains on one GPU box. It's fine for experiments, but it has no reproducibility, and a crash loses days of work.

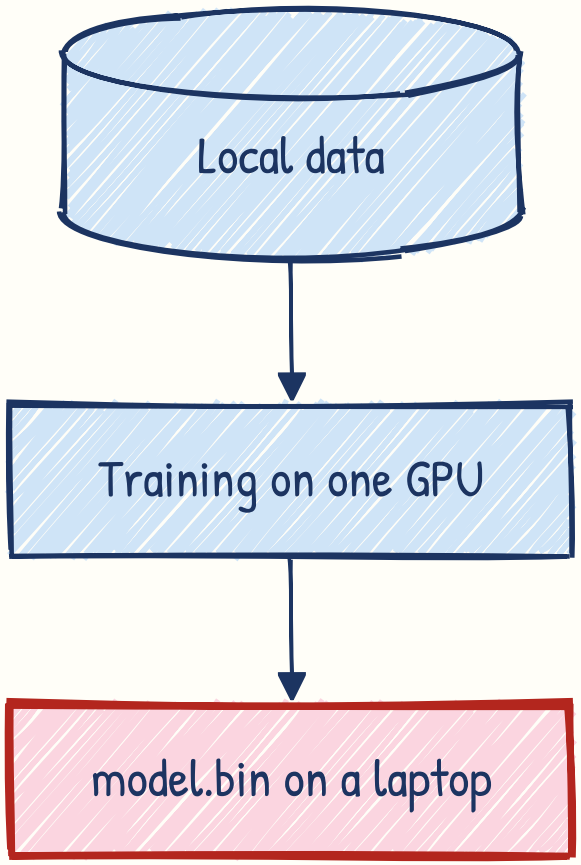


Figure 5. Baseline. Fine for a prototype, but one failure loses everything, and nobody can reproduce the result.

Improved: an orchestrated pipeline with checkpoints. Versioned data, and saved progress. New problem: **the model now exists, but Python training code and your Java services are separate worlds**, and if they compute features differently, the model quietly misbehaves.

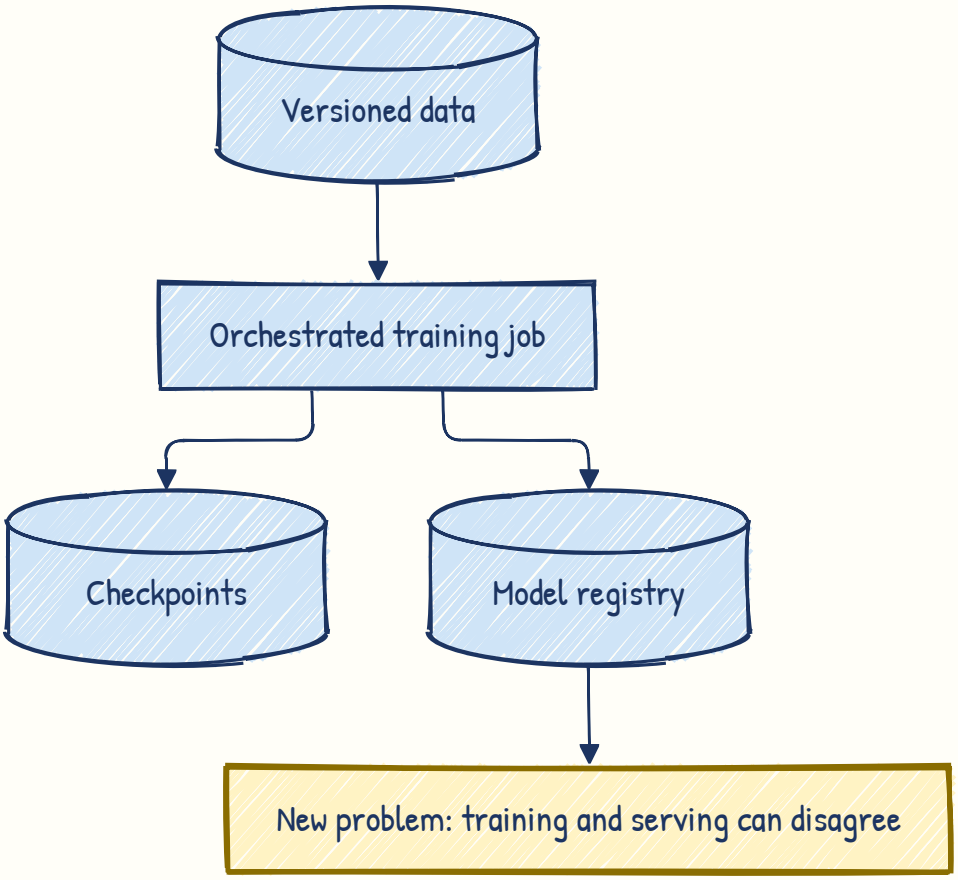


Figure 6. Improved. Training is repeatable and resumable, but the hand-off to the serving system is now the risk.

Production: a clean serving contract.

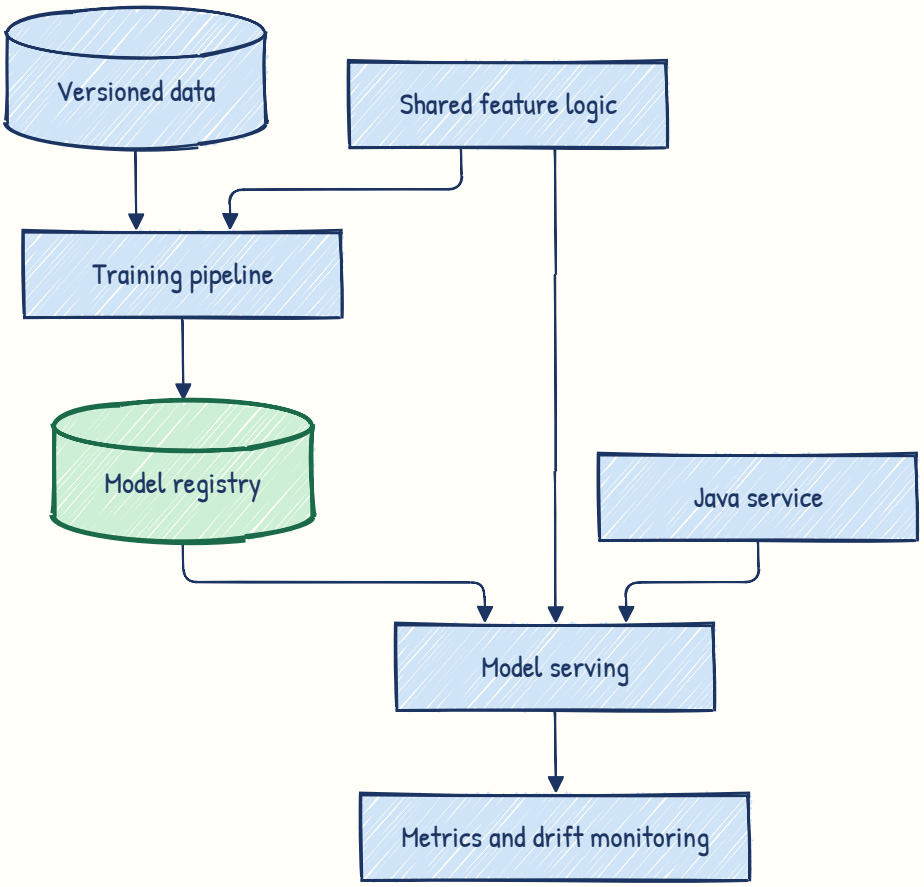


Figure 7. Production. The registry holds versioned artifacts, and shared feature logic keeps training and serving consistent. Java services call one stable API.

Component	Why it exists	How it communicates
Checkpoints	GPU runs can fail after days	Written to object storage during the run
Model registry	Records which model is live and allows rollback	Serving loads a named version
Shared feature logic	Prevents training-serving skew	Same library or store on both sides (HLD M21C)
Model serving	Keeps GPU work away from business services	gRPC or HTTP, with tight timeouts

Failures

Failure	Containment
Loss becomes `NaN`	Stop early, lower the learning rate, clip gradients, then resume from a checkpoint
GPU node dies	Resume from the last checkpoint
Feature mismatch	Shared logic, and compare training and serving distributions
New model is worse	Canary, then roll back through the registry
Slow inference	Batch requests, quantize the model (AI11 L2), or use a smaller model

Java options

Keep it simple:

→ **Call a hosted LLM:** the normal case for this course (AI08).

→ **Serve a small exported model in the JVM:** ONNX Runtime has a Java API.

→ **Run models in a framework from Java:** the Deep Java Library (DJL) exists for this. Most teams still train in Python.

Trade-offs

Decision	You gain	It costs
Deeper or wider network	More capacity	More data, compute and overfitting risk
Higher learning rate	Faster progress	Instability
Dropout, weight decay	Better generalization	Slower training, more settings
Larger batch	Efficient GPU use, smoother steps	Memory
Neural network vs gradient boosting	Handles raw text and images	Often worse on tables, and harder to explain

When not to use a neural network: small tabular data, when explainability is a strict requirement, or when a hosted model already solves your problem.

Common mistakes

1. **Thinking neural networks work like brains.** The analogy is loose. They're differentiable math functions.
2. **Using linear activations in hidden layers.** All the layers collapse into one.
3. **Ignoring a `NaN` loss.** Check learning rate and data first.
4. **Judging on training loss.** Watch validation loss for overfitting.
5. **Reaching for deep learning on small tables.** Gradient boosting often wins with less effort.
6. **Treating "more layers" as automatically better.** Without residual connections and normalization, very deep networks train poorly.

What to say in an interview

For "explain backpropagation", don't start with calculus. Say: "It's how we find each weight's share of the error efficiently. We run a forward pass, then apply the chain rule backward, reusing results. Rather than one pass per weight, it costs about one extra pass, which is what makes billions of parameters feasible." Then offer the gradient check as how you'd verify it.

Certification lens

How this lesson maps to CCAR-F

Nothing here is tested. Model internals, training and weights are all explicitly out of scope. This lesson exists for your production goal, and for AI02. If you're short of time before the exam, skim it.

These are original practice questions for understanding, not exam simulations.

Q Q1 · Output prediction · Medium. A one-neuron model has w = 1, x = 1, no bias and target 1. Using a sigmoid and a learning rate of 0.5, what's the new weight after one step? Use σ(1) = 0.7311.

A Gradient = (0.7311 − 1) × 1 = −0.2689. New w = 1 − 0.5 × (−0.2689) = **1.1345**. The weight rises, pushing the prediction closer to 1.

Q Q2 · Troubleshooting · Medium. Training loss falls steadily and validation loss starts rising after epoch 12. What's happening, and what are two fixes?

A Overfitting: the model is memorizing the training data. Stop early around epoch 12, and add regularization such as dropout or weight decay. More data would also help.

Q Q3 · Assertion and reasoning · Hard. Assertion: replacing all hidden activations with a linear function removes the model's ability to learn XOR. Reason: a composition of linear functions is itself linear. Choose: (A) both true, and R explains A; (B) both true, and R doesn't explain A; (C) A true, R false; (D) A false, R true.

A **A.** Stacked linear layers collapse to one linear function, which can only draw one straight boundary. XOR needs more than that, as the demo's linear model showed, stuck at 0.50.

Revision - Lesson 3

→ A neuron computes a weighted sum, adds a bias, and applies a non-linear activation.

→ Without non-linearity, stacked layers collapse into one linear function, which can't learn XOR.

→ Training: forward pass, loss, backpropagation, optimizer update. Repeat.

→ Cross-entropy is −ln(p of the correct answer). LLMs use it too.

→ Backpropagation finds all gradients in about one backward pass, using the chain rule.

→ Learning rate: too low is slow, and too high diverges, possibly to NaN.

→ Mini-batches are the norm. Adam combines momentum with per-weight scaling.

→ Deep chains shrink gradients. ReLU, normalization and residual connections keep training stable.

→ Regularization (weight decay, dropout, early stopping) fights overfitting.

→ CNNs share weights across positions. Softmax outputs probabilities.

Your notes